



UNIVERSITÀ DEGLI STUDI DI SALERNO

Relazione di sviluppo e progettazione dell'applicazione UniLife

Mobile Programming - A.A. 2025/26

Gruppo 15

Emiddio - 0612708848

Matteo IV Buono - 0612708893

Gaetano Alessio Forino - 0612708777

Rosa Genovese - 0612709529

Lucia Canzolino - 0612708883

Mobile Programming - Anno Accademico 2025/2026

Sommario

UniLife è un'applicazione mobile cross-platform sviluppata in Flutter che assiste lo studente universitario nella gestione di *corsi*, *esami*, *sessioni di studio* e *attività*, con un timer Pomodoro integrato e una dashboard di sintesi. L'app è progettata per funzionare **offline** e con un singolo utente, persistendo i dati in un database SQLite locale.

Indice

Sommario	1
1 Introduzione	3
1.1 Obiettivi	3
1.2 Struttura del documento	3
2 Progettazione Database	4
2.1 Schema Concettuale UML	4
2.2 Schema Logico	5
2.2.1 Vincoli non esprimibili nello schema logico	5
3 Architettura software	5
3.1 Stack tecnologico	5
3.2 Organizzazione del codice	6
3.3 Pattern adottati	6
4 Implementazione	7
4.1 Persistenza locale	7
4.2 State management	7
4.3 Notifiche locali	7
5 Interfaccia utente	7
5.1 Design system	7
5.2 Dashboard	7
5.2.1 Sessioni di Studio	8
5.3 Corsi	9
5.4 Esami	10
5.5 Focus / Pomodoro	12
5.6 Impostazioni	12
6 Scelte progettuali peculiari	13
6.1 Da cinque a quattro tab	13
6.2 Il voto non appartiene al corso	13
6.3 Doppio entry point per la pianificazione delle sessioni	13
6.4 Esami standalone	14
7 Funzionalità considerate ma non implementate	14
8 Limitazioni note	14
9 Conclusioni	14

1 Introduzione

Il nostro team ha lavorato per la realizzazione di un'applicazione per dispositivi mobili per risolvere uno dei più grandi problemi per gli studenti: **la dispersione delle varie attività da svolgere** come esami, corsi o sessioni di studio.

1.1 Obiettivi

- Centralizzare in un'unica app le attività principali della vita accademica (corsi, esami, sessioni, task).
- Funzionare **offline**, single-user, con persistenza locale.

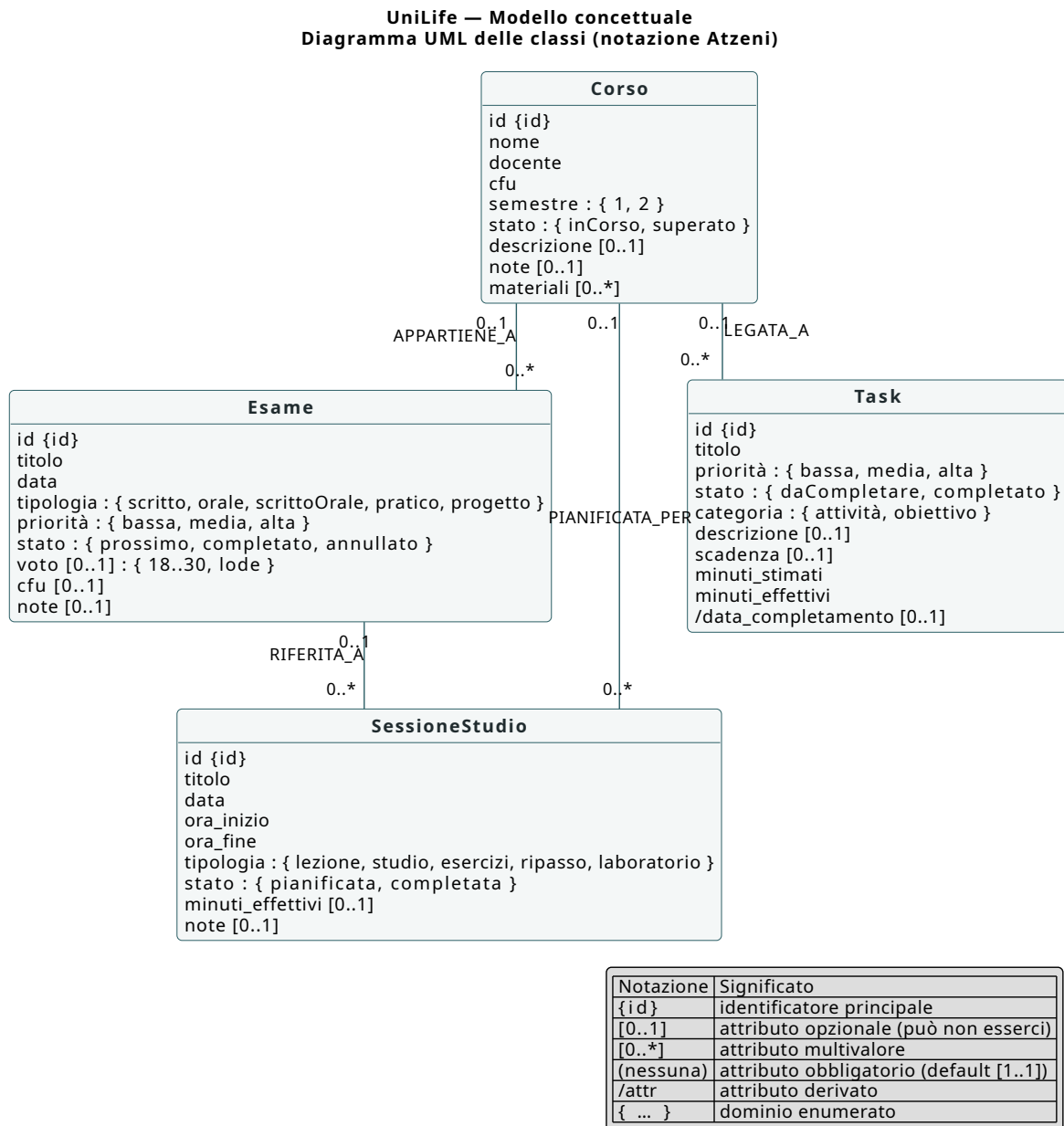
1.2 Struttura del documento

Il documento è organizzato in tre macro-parti:

- Progettazione del database
- Progettazione e Realizzazione delle schermate dell'applicazione
- Verifiche e considerazioni finali

2 Progettazione Database

2.1 Schema Concettuale UML



2.2 Schema Logico

```

COURSES(id PK, nome, docente, cfu, semestre, stato,
         descrizione, note, materiali)

EXAMS(id PK, title, course_id FK->COURSES, cfu, date, type,
       priority, status, grade, notes)

STUDY_SESSIONS(id PK, title, course_id FK->COURSES,
                exam_id FK->EXAMS, date, start_time,
                end_time, type, is_completed,
                actual_minutes, notes)

TASKS(id PK, title, description, course_id FK->COURSES,
       priority, status, due_date, estimated_minutes,
       actual_minutes, completed_at, is_goal)

```

2.2.1 Vincoli non esprimibili nello schema logico

- **Esami standalone:** vincolo inter-attributo su EXAMS $\rightarrow$ cfu IS NOT NULL $\Leftrightarrow$ course_id IS NULL
- **Vincoli referenziali:** tutte le FK hanno ON DELETE SET NULL
- **Vincoli di dominio applicativi:**
 - COURSES.cfu è compreso tra [1,30], COURSES.semestre assume valori in {1,2}
 - EXAMS.grade assume valori in [18,31] (31 = lode), EXAMS.status in {prossimo, completato, annullato}
 - TASKS.status assume valori in {daCompletare, completato}

3 Architettura software

3.1 Stack tecnologico

Tabella 1. Tecnologie principali e motivazione della scelta.

Ambito	Pacchetto	Motivazione
State mgmt	provider	Allineato alle slide del corso
Routing	go_router	Passaggio esplicito di parametri
DB locale	sqflite	Query relazionali, indici, FK
Preferenze	shared_preferences	Tema, nome utente, preset
Notifiche	flutter_local_notifications	Notifica fine Pomodoro
Calendario	table_calendar	Vista mensile localizzata
Date/intl	intl	Formattazione it_IT

3.2 Organizzazione del codice

```
lib/
+-- main.dart
+-- app.dart
+-- core/
|   +-- constants/   app_constants.dart
|   +-- router/      app_router.dart
|   +-- theme/       app_colors.dart, app_theme.dart, theme_preset.dart
|   \-- utils/       date_utils.dart, validators.dart
+-- data/
|   +-- datasources/  database_helper.dart
|   +-- models/       enums, course, exam, study_session, task
|   \-- repositories/ course, exam, session, task (4 file)
+-- state/           course, exam, session, task, pomodoro, stats, theme
+-- services/        notification_service.dart, url_service.dart
+-- features/
|   +-- shell/        home_shell.dart
|   +-- dashboard/    dashboard_screen + 3 widget
|   +-- courses/      list, detail, form + course_card
|   +-- exams/        list, detail, form, calendar + exam_tile
|   +-- tasks/        task_form + task_tile
|   +-- planning/     session_form + weekly_planner
|   +-- focus/        focus_screen + pomodoro_timer
|   \-- settings/     settings_screen.dart
\-- shared/widgets/   empty_state, confirm_dialog, app_snackbar,
                     status_chip, search_field
```

Tabella 2. Responsabilità delle cartelle del package lib/.

Cartella	Responsabilità
core/	Tema, router, costanti, utility trasversali.
data/	Modelli, database helper, repository CRUD.
state/	Sette ChangeNotifier (App state).
services/	Wrapper sulle API native (notifiche, URL).
features/	Schermate organizzate per area funzionale.
shared/	Widget riusabili in più feature.
main.dart	Entry point: init DB, notifiche, runApp.
app.dart	Root widget: MultiProvider + MaterialApp.router.

3.3 Pattern adottati

Repository + Provider + ChangeNotifier: la UI parla solo con i Provider, i provider parlano solo con i Repository, i repository incapsulano l'SQL. Garantisce separazione delle responsabilità (SRP) e testabilità.

```
1 class CourseProvider extends ChangeNotifier {
2   CourseProvider({CourseRepository? repository})
3     : _repo = repository ?? CourseRepository();
4
5   Future<void> load() async {
6     _loading = true; notifyListeners();
7     _all = await _repo.getAll();
8     _loading = false; notifyListeners();
```

```
9   }  
10  }
```

Listing 1. Estratto da `course_provider.dart`

4 Implementazione

4.1 Persistenza locale

Tutti i dati prodotti dall'utente devono restare sul dispositivo, e sopravvivere al riavvio dell'app e poter essere interrogati anche in assenza di connessione di rete.

Per i dati strutturati relazionali come esami, task e corsi si è adottato SQLite e la classe helper associata `DatabaseHelper` (la cui unica istanza vive per l'intero ciclo dell'app) per garantire affidabilità, portabilità e prestazioni ottimali, mentre per le preferenze utente nella sezione `settings`, si è deciso di gestirle come una semplice hashmap in `shared_preferences`.

4.2 State management

Sono definiti sette `ChangeNotifier`: `CourseProvider`, `ExamProvider`, `SessionProvider`, `TaskProvider`, `PomodoroProvider`, `StatsProvider`, `ThemeProvider`. Sono montati in `app.dart` tramite `MultiProvider`; `StatsProvider` è un `ChangeNotifierProxyProvider3` che dipende dai tre provider di dominio.

4.3 Notifiche locali

Il timer Pomodoro emette una notifica di fine sessione tramite `flutter_local_notifications`. L'API v21 richiede parametri *tutti* nominati (`initialize(settings: ...)`, `show(id:, title:, body:, notificationDetails:)`).

5 Interfaccia utente

5.1 Design system

L'app aderisce a Material 3 con un `ColorScheme` generato da un *seed color*. La palette di default è petrolio + arancio caldo; sono disponibili cinque temi stagionali (Fig. 10).

5.2 Dashboard

La dashboard fonde quattro elementi: il saluto personalizzato, le scadenze imminenti con anello di completamento delle attività di giornata, le card statistiche, la lista dei task del giorno con checkbox di completamento, e la sezione *Prossime sessioni* con accesso rapido al calendario.

Ogni sessione mostrata in Dashboard è cliccabile per la modifica.

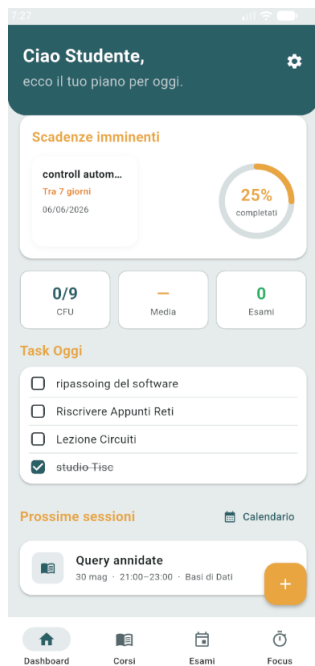


Figura 1. Dashboard in tema chiaro.

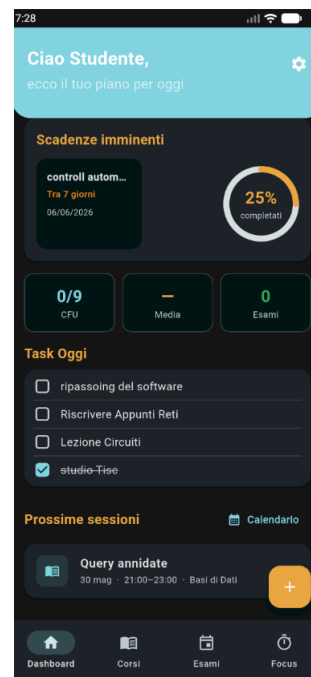


Figura 2. Dashboard in tema scuro.

5.2.1 Sessioni di Studio

Le sessioni di studio non hanno un tab dedicato nella bottom navigation bar ma sono integrate in vari punti dell'interfaccia, coerentemente con la scelta di mantenere il prototipo a quattro tab.

Sul **Calendario** le sessioni compaiono come marker sui giorni in cui sono pianificate o sono state svolte; selezionando un giorno, la lista degli eventi mostra un widget tile dedicato per ciascuna sessione con orario, tipologia e indicatore di completamento. Un pulsante *+ Sessione* permette di inserire una nuova sessione di studio aprendo il form precompilato in cui è possibile specificare orario di inizio e di fine.

Sulla **Dashboard**, la sezione *Prossime sessioni* elenca fino a cinque sessioni della settimana corrente: ogni voce mostra il nome della sessione (gli argomenti principali da studiare), data e ora di inizio e il corso associato.

Se l'utente avvia una sessione Pomodoro dal tab **Focus**, al termine del timer (o all'interruzione anticipata con salvataggio dei minuti svolti) il sistema crea automaticamente una nuova sessione di studio nel database, ereditando dal Pomodoro il corso, il task collegato e i minuti effettivi.

5.3 Corsi

Il tab **Corsi** è il libretto digitale dello studente: presenta i corsi organizzati in due schede mutuamente esclusive, corsi correnti e terminati, selezionabili tramite una *TabBar* in testa alla schermata.

Sotto la *TabBar* è presente una barra di ricerca che consente di filtrare i corsi per nome del corso o del docente.

Ciascun corso è rappresentato come *card* con il nome in evidenza, docente, semestre, CFU come badge e una barra di avanzamento basata sul rapporto fra esami superati e totali del corso. Nella sezione **Attività collegate** qualunque task associato ad un corso sarà visibile comodamente dall'interfaccia, consentendo di avere una visione generale delle attività svolte e da svolgere.



Figura 3. Lista corsi con tab Correnti/Terminate.

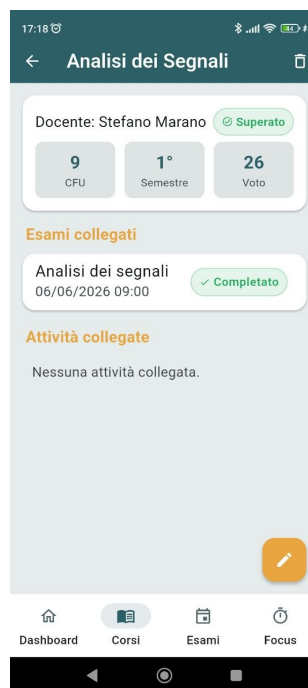


Figura 4. Dettaglio del corso con esami associati.

Il tap sulla card apre il **Dettaglio del corso** che aggrega tre blocchi:

- L'anagrafica completa del corso
- La lista degli esami associati al corso
- Le attività associate al corso

Il bottone di modifica nell'angolo consente di aprire il form di edit ed eventualmente di aggiungere link ipertestuali a materiale esterno utile per lo studio e l'approfondimento degli argomenti del corso.

5.4 Esami

Il tab *Esami* è il registro degli appelli dello studente: presenta gli esami raggruppati per data e filtrabili tramite una **TabBar** a quattro voci — *Tutti*, *Prossimo*, *Completato* e *Annullato* — per una navigazione mirata in base allo stato dell'appello.

Tramite l'icona del *Calendario* nell'angolo superiore destro è possibile accedere a una vista mensile in cui vengono visualizzati tramite *marker* gli esami, le sessioni di studio e i task pianificati.

Ciascun esame nella lista è rappresentato come una riga strutturata che evidenzia:

- La data dell'appello espressa in giorno e mese;
- Il nome dell'esame, il corso di afferenza e l'orario previsto;
- Un *chip* di stato (*Completato*, *Prossimo*, *Annullato*) che ne attesta l'avanzamento.

La selezione di un esame apre il *Dettaglio esame*, che raggruppa tutte le informazioni e le azioni disponibili:

- **Dati anagrafici:** corso, docente, data dell'appello, CFU, tipologia di prova e note.
- **Cambio stato inline:** pulsanti rapidi per modificare lo stato tra *Prossimo*, *Completato* e *Annullato*.
- **Registra voto:** sezione visibile solo a esame *Completato*, consente l'inserimento del voto nell'intervallo 18–30L aggiornando automaticamente la media ponderata in dashboard.
- **Modifica ed eliminazione:** il bottone nell'angolo superiore destro apre il form di modifica o avvia la rimozione definitiva previa conferma in un *dialog* di sicurezza.



Figura 5. Lista esami con TabBar di stato.

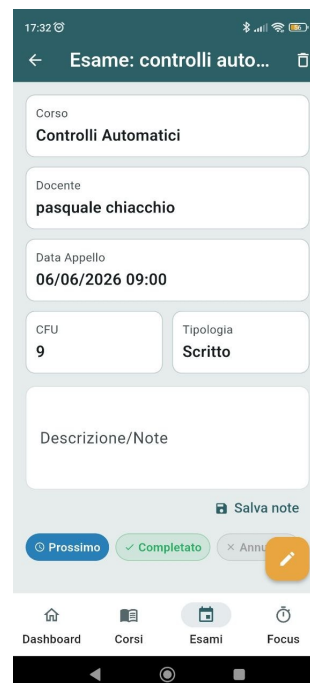


Figura 6. Dettaglio esame con registrazione voto.



Figura 7. Calendario mensile con eventi.

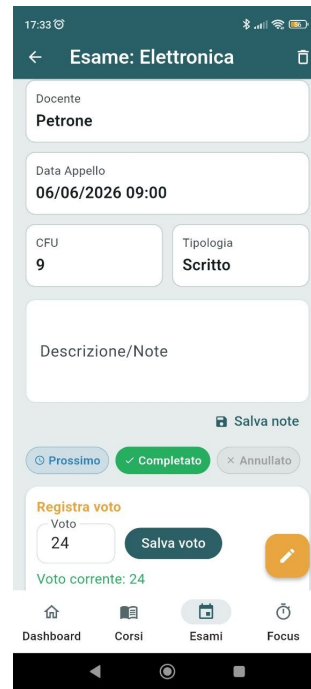


Figura 8. Dettagli esame convalidato a libretto con voto.

5.5 Focus / Pomodoro

Il tab **Focus** presenta un timer circolare a countdown al centro dello schermo, progettato per guidare lo studente attraverso la tecnica Pomodoro.

Sotto il timer sono disponibili tre preset di durata selezionabili — *5 min*, *25 min* e *15 min* — rispettivamente pensati per pause brevi, sessioni standard e pause lunghe; la durata attiva è evidenziata visivamente con un segno di spunta.

I pulsanti **Avvia** e **Stop** controllano il ciclo del timer. Al termine del countdown, il sistema invia una notifica locale al sistema operativo, attiva anche con l'app in background.

Nella sezione *A cosa stai lavorando?* lo studente seleziona tramite menu a tendina il corso e, facoltativamente, il task da tracciare. Al completamento, questi dati vengono associati automaticamente alla sessione registrata.

Se il timer viene interrotto anticipatamente, una finestra di conferma propone due opzioni: **Salva e termina**, che salva i minuti parziali, oppure **Scarta**, che annulla la sessione senza salvare dati.

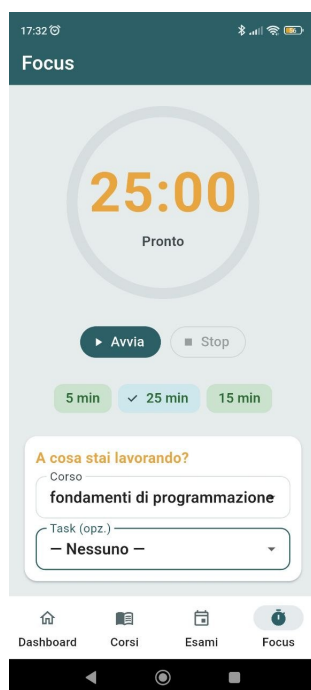


Figura 9. Schermata Focus con timer Pomodoro circolare.

5.6 Impostazioni

La schermata **Impostazioni**, raggiungibile tramite la Dashboard, raccoglie le preferenze di personalizzazione dell'applicazione, organizzate in tre sezioni.

La sezione **Profilo** consente di inserire il proprio nome, che sostituisce il segnaposto generico nella Dashboard.

La sezione **Aspetto** espone un toggle per il **Tema scuro** e un'opzione *Usa tema di sistema*, che delega la scelta al sistema operativo adeguandosi automaticamente alle preferenze dell'utente.

La sezione **Tema stagionale** presenta cinque palette cromatiche selezionabili — *Classico*, *Halloween*, *Natale*, *Pasqua* e *Primavera* — applicate istantaneamente all'intera applicazione e persistite tra le sessioni.

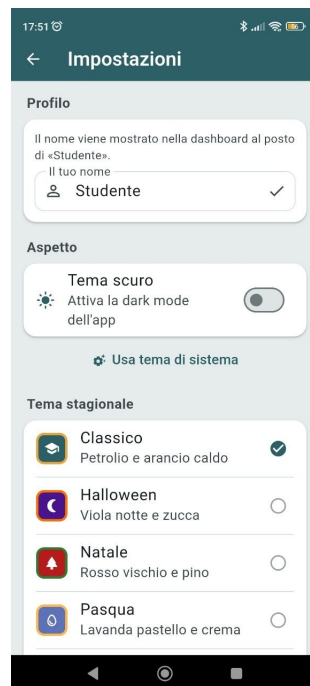


Figura 10. Sezione Impostazioni con profilo, dark mode e temi stagionali.

6 Scelte progettuali peculiari

6.1 Da cinque a quattro tab

Il documento dei requisiti prevedeva cinque tab (Corsi, Esami, Pianificazione, Attività, Statistiche). L'analisi del prototipo a bassa fedeltà ha suggerito un consolidamento a quattro tab (Dashboard, Corsi, Esami, Focus): la Dashboard accorpa statistiche, task del giorno e sessioni di studio; la Pianificazione viene raggiunta dal calendario; l'Attività ha il proprio + ma vive sulla Dashboard.

6.2 Il voto non appartiene al corso

Nel modello iniziale il voto era un attributo del CORSO. È stato rimosso perché ritenuto ambiguo nel contesto in cui viveva: infatti causava anomalie di aggiornamento quando lo studente rifiutava il voto. Il voto vive ora solo su ESAME; il voto del corso è derivato a runtime dall'ultimo esame con stato *completato*.

6.3 Doppio entry point per la pianificazione delle sessioni

La traccia richiede una funzionalità di pianificazione esplicita delle sessioni di studio. Si è scelto di offrire *due* flussi complementari accessibili da contesti diversi:

- **Pianificazione in anticipo:** dal Calendario, il pulsante *+ Sessione* che precompila il giorno selezionato. Pensata per lo studente che vuole organizzare la settimana
- **Sessione Pomodoro:** dal Focus, il timer Pomodoro al termine salva una sessione con i minuti effettivi.

6.4 Esami standalone

Per consentire la registrazione retroattiva di esami senza obbligare la creazione del corso, `ESAME.course_id` è stato reso opzionale e si è aggiunta la colonna `cfu`, valorizzata solo per gli esami non legati ad alcun corso.

7 Funzionalità considerate ma non implementate

In fase di analisi sono state considerate alcune funzionalità ritenute interessanti ma impossibili da implementare a causa di fattori di tempo e di incoerenza con le richieste esplicite della traccia:

- Integrazione applicazione con le API dell'università per poter aggiornare e visualizzare in tempo reale le informazioni critiche come corsi o esami
- Notifiche di avviso per task o esami in scadenza
- Suggerimento automatico di attività in base agli esami imminenti.
- Sezione tab *Altro* in cui possono essere raccolte altre funzionalità (es. Statistiche) senza violare il design a 4 tab originale.

8 Limitazioni note

- **Single-user:** l'app gestisce un profilo utente locale e non permette la presenza di più utenti sullo stesso dispositivo.
- **Calendario nel tab Esami:** si è scelto di non promuovere il calendario come quinto tab per non alterare la struttura a quattro tab del prototipo.
- **Test manuali:** per problemi di logistica, è stato impossibile produrre test automatizzati con la suite di Flutter e il team si è impegnato a utilizzare una tecnica di feedback su campo reale testando l'applicazione per eventuali bug più evidenti per rendere l'esperienza più fluida e interattiva possibile.

9 Conclusioni

In conclusione, l'applicazione è stata sviluppata cercando di rispettare a pieno le specifiche richieste, valorizzando eventualmente i requisiti che abbiamo ritenuto essere più rilevanti ai fini dell'esperienza utente. Le scelte progettuali più formative sono state quelle dettate da incoerenze emerse durante l'implementazione:

- la rimozione del voto dal CORSO
- l'introduzione degli esami standalone
- il passaggio da cinque a quattro tab

Ogni scelta ha richiesto di mettere in discussione il modello iniziale e di documentare esplicitamente la motivazione del cambiamento, rispecchiando il flusso reale dello sviluppo software.

Riferimenti bibliografici

- [1] Flutter SDK, <https://flutter.dev>.
- [2] R. Roussel, *Provider package*, pub.dev, <https://pub.dev/packages/provider>.
- [3] T. Pham, *sqflite package*, pub.dev, <https://pub.dev/packages/sqflite>.
- [4] Google, *Material Design 3*, <https://m3.material.io/>.
- [5] Paolo Atzeni, *Basi di Dati*, sito ufficiale